

Outlay — Competitive battlecards

Field reference for positioning Outlay against the four categories that touch AI-spend optimization. Grounded in [docs/market-analysis-ai-spend-governance.md](#) §11. **Keep it honest** — the fastest way to lose a technical buyer is an overclaim they can disprove in one click. Lead with where we're genuinely different, concede where a competitor is genuinely better fit, and disqualify early when we're not the right tool.

One-sentence positioning: Outlay is the work-attribution + forecast + governance layer for AI spend — it answers "what is AI costing us per team, feature, and engineer," forecasts it on your own delivered work, and governs it — read-only, BYOK, metadata-only. Everyone else stops at team/project dashboards, sits in your request path, or optimizes cloud infra you don't run.

The four moats (repeat these; they're what no competitor leads with): 1. **Work-item attribution** — token → ticket → engineer → feature, via the tracker join. Not just team/project. 2. **Forecast back-tested on your own delivered work** — leave-one-out error on your history, not a vendor benchmark. 3. **Metadata-only / BYOK / read-only** — prompts, outputs, and keys never leave your box. The compliance wedge. 4. **Budget governance + pacing** as a first-class workflow — programs, projected-breach dates, on-/off-track ratings — not a chart.

1. vs. AI Cost Visibility / FinOps (nearest competitors)

Vantage · CloudZero · Finout · Datadog Cloud Cost · Flexera FinOps-for-AI · usage.ai · Amnic

Their pitch: "Connect your OpenAI/Anthropic/cloud billing and see all your AI spend in one dashboard — attributed to team, project, and customer, with anomaly alerts."

Where they genuinely win (concede these): - Mature, broad **cloud** cost coverage (EC2/S3/k8s) with AI bolted on — if the buyer's pain is 90% cloud infra and 10% LLM, they fit better. - Established FinOps-team workflows, large integration catalogs, SOC 2 today. - Unit-economics / cost-per-customer dashboards are polished.

Where Outlay wins: - **Granularity:** they attribute to team / project / cost-center; we reach the **work item** — the ticket, feature, and engineer — through the tracker join. "What did shipping the checkout refactor cost?" has an answer in Outlay and not in a Vantage dashboard. - **Forecast honesty:** we forecast the backlog's cost and **back-test the error on the customer's own completed work**. Most visibility tools show history + a naive trend; few quote a measured accuracy number on your data. - **Posture: metadata-only / BYOK / read-only** is a first-class design choice and a compliance unlock; they're cloud-billing integrations that ingest more. - **Governance:** program budgets with **projected-breach dates** and **earned-value on-/off-track ratings**, not just alerts on a chart.

Discovery questions (plant the landmines): - "Can your current tool tell you what a specific feature or ticket cost to build — or only what a team or project spent?" - "When you forecast next quarter's AI budget, what's the measured error of that forecast on your own past work?" - "Does your spend tool ingest prompt/response content, or only metadata? What did security say about that?"

Objection → response: - "We already have Vantage/CloudZero." → "Great for cloud and team-level rollups — keep it. Outlay sits one level deeper on the AI line: ticket/feature attribution and a forecast you can defend to finance. Teams run both; we're the work-attribution layer they don't have." - "Isn't this just another dashboard?" → "A dashboard tells you what happened. Outlay tells you what each piece of work cost, what the backlog will cost, and flags the breach before month-end. It's a system you act on, not a report you read."

Trap to avoid: don't argue "we're cheaper visibility." We're not competing on visibility breadth — we win on depth of attribution + forecast + governance. Stay there.

One-liner: They show you team-level spend. We show you what each feature cost and what the next one will — and stop the overrun.

2. vs. LLM Gateways / Routers

LiteLLM · Portkey · OpenRouter · Cloudflare AI Gateway · TrueFoundry · Helicone · Martian

Their pitch: "Route every request through us — we cache, fall back, route to cheaper models, and enforce per-key/per-team budget caps in-path. Cut 20–40% off your bill."

Where they genuinely win (concede these): - **Hard enforcement:** they can block a request at the budget ceiling because they're in the path. If the buyer needs a literal kill-switch, that's a gateway job. - **Request-level optimization:** caching and cheaper-model routing produce real, immediate savings. - Loved by **platform engineering** — it's infrastructure they already own.

Where Outlay wins: - **Posture:** gateways sit **in the request path**; Outlay is **read-only, out-of-path**. For regulated / security-sensitive buyers, "nothing new touches the prompt or the key" is the entire reason they'll talk to us. A gateway is a new dependency in the critical path and a new place prompts flow through. - **Buyer & altitude:** gateways serve platform eng with infra knobs; Outlay serves **finance + eng leadership** with attribution, forecast, and governance. Different question — "is this request cheap?" vs. "what is AI costing the business and is it on budget?" - **Commitment/portfolio optimization:** gateways cut per-request cost; they don't tell you whether to take a **committed-spend discount or provisioned throughput** across your whole run-rate. Outlay does (the Commitments surface). - **Attribution:** routing keys ≠ work items. We join to the ticket/feature; they meter the key.

Discovery questions: - "Are you comfortable putting a new service in the path of every model call — and routing prompts through it — or do you need spend control that stays read-only?" - "Your gateway cuts per-request cost. Who's deciding whether you should be on on-demand vs. a committed-spend discount for your steady volume?" - "Can your gateway attribute spend to a Jira ticket or a shipped feature, or only to an API key?"

Objection → response: - "LiteLLM/Portkey already caps our budgets." → "In-path caps are great for enforcement — keep them. They can't tell you what each feature cost, forecast the backlog, or size a commitment. And many teams can't put a gateway in the path for compliance reasons — that's exactly who we serve. We integrate with the gateway, we don't replace it." - "We'd rather cut cost in real time than measure it." → "Do both — they're complementary. But the biggest lever isn't per-request; it's not overpaying on-demand for steady volume and not building features whose cost no one owns. That's measurement + governance + commitment sizing."

Trap to avoid: don't position as an enforcement tool against gateways — we'll lose on enforcement. Position on **posture (read-only), altitude (finance/governance), and commitment optimization**. Note we have an opt-in enforcement endpoint as a bridge, but it's not the pitch.

One-liner: Gateways cut the price of a request from inside the path. We govern the whole AI budget from outside it — and never touch a prompt.

3. vs. Cloud Commitment Optimizers

ProsperOps (Flexera, acq. Jan 2026) · Zesty · nOps

Their pitch: "We autonomously ladder Reserved Instances / Savings Plans / CUDs to maximize your discount on cloud GPU and compute — billions in managed savings, 30-50% on GPU."

Where they genuinely win (concede these): - For **self-hosted GPU / cloud-instance** spend (you run the models on EC2/GKE), they're excellent and autonomous. - Proven savings engines with real scale and a hands-off RI/SP ladder. - Flexera now bundles this with FinOps-for-AI — a consolidation threat to watch.

Where Outlay wins: - **Different layer:** they optimize **infrastructure you operate**; most enterprises consume **managed model APIs** (Anthropic, OpenAI, Bedrock, Azure OpenAI) they don't run on reservable instances. Cloud RI laddering doesn't touch that bill. - **Model-API commitment intelligence:** committed-spend discounts and **provisioned throughput (PTUs)** at the model layer is open white space — Outlay's Commitments surface sizes it. Cloud optimizers don't cover it. - **The attribution moat:** our commitment recommendation is built on **work-attributed steadiness** — we know which workloads are steady enough to commit because we joined them to the work. An infra-only optimizer can't make that call. - **Posture:** read-only/advisory; we recommend, you execute with the vendor. No autonomous changes to your account.

Discovery questions: - "Are you running models on your own GPU instances, or consuming managed APIs (Anthropic/OpenAI/Bedrock/Azure)? Because RI laddering only helps the first." - "Who's sizing your committed-spend discount or PTU reservation at the model-API layer — and are they basing it on which workloads are actually steady?"

Objection → response: - "ProsperOps already optimizes our commitments." → "For cloud instances, yes. Your **model-API** spend — the Anthropic/OpenAI bill — isn't reservable cloud capacity; it needs committed-spend and PTU sizing based on your usage steadiness. That's what Outlay does, and we feed it the work-attributed data they don't have." - "Flexera does both now." → "They're assembling it — which is the signal this layer matters. Today their strength is cloud infra + visibility; the model-API commitment layer driven by work-item steadiness is still open, and that's our wedge. We'd rather you have it before they finish stitching it."

Trap to avoid: don't claim we replace ProsperOps for cloud RIs — we don't. We're the **model-API** commitment layer; cloud-instance buyers may genuinely need both.

One-liner: They reserve the GPU instances you run. We size the commitment on the model APIs you consume — using which work is actually steady.

4. vs. Provider-native dashboards & "we'll build it ourselves"

OpenAI/Anthropic usage pages · Azure/AWS cost tools · an internal script

Their pitch (often the real competitor): "The provider console shows our usage, and we have a data analyst who exports it to a spreadsheet."

Where Outlay wins: - **Cross-provider:** provider pages are single-vendor and aggregated. Real teams are on 3-6 providers/tools; no native view unifies them. - **The join is the hard part:** mapping token spend to a ticket/engineer/feature requires correlating provider usage + key identity + git branch + tracker — exactly the engine Outlay is. A spreadsheet can't sustain it, and the internal script rots the moment a model id or export format changes. -

Forecast + governance: no provider page forecasts your backlog or paces a program budget. - **Maintenance cost:** "build it ourselves" is a standing eng cost (rate-card drift, new providers, attribution logic) that's never the team's core product.

Discovery questions: - "How many AI providers/tools are you on today? How do you get one unified, attributed view?" - "Who maintains the attribution script when a provider changes its export or you add a tool — and is that the best use of that engineer?" - "Can your spreadsheet forecast next quarter's AI spend with a measured error bar?"

Objection → response: - "We can build this internally." → "You can build a v1 dashboard. The cost is the upkeep — the spend→work join across changing providers, the forecast back-test, the governance workflow. We've built and maintain that so your engineers ship product instead. And it's metadata-only, so there's nothing to get cleared by security."

One-liner: The provider page shows one vendor's tokens. The attribution join across all of them — kept working as the stack changes — is the product.

Honest disqualifiers — when to walk

Naming these builds trust and saves cycles: - **Spend is mostly self-hosted GPU on cloud instances** → ProsperOps/Zesty/nOps fit better; we're the model-API layer. - **The only need is a hard in-path kill-switch** → a gateway (LiteLLM/Portkey) is the right primary tool; we complement, not replace. - **AI spend is still tiny / a few \$20 seats** → no consumption bill to attribute yet; revisit when usage scales. - **No work tracker / no branch hygiene** → our differentiating join degrades to team-level; we still help, but our sharpest edge is dull. Be honest about fidelity.

The summary table

Category	They optimize	We optimize	Our edge in one phrase
Visibility/ FinOps	Team/project spend, cloud-first	Work-item spend + forecast + governance	Depth of attribution, measured forecast
Gateways/ routers	Per-request cost, in-path	Whole-budget governance, out-of-path	Read-only posture + commitment sizing
Cloud optimizers	Cloud GPU reservations	Model-API commitments by steadiness	The work-attributed steadiness signal
Provider/DIY	One vendor's raw usage	Unified, attributed, maintained	The cross-provider join, kept alive